

Moonshot Inc — Products / R&D / Capacity: Implementation Spec

The engine work behind the authored content. This is the build-side companion to `06_PRODUCTS_RD_CAPACITY_AUTHORING.md` (content) — it specifies the schemas, the math, the state, the loop wiring, the UI surfaces, and validation the code must implement. Plus the three reported bugs as a fix list at the end.

Conventions (match the existing codebase): money in `$M`; time in **weeks**; ids global snake_case; the engine is **pure and deterministic** (seeded RNG, no `Math.random`); cross-ref problems push to `ContentDB.warnings` (don't crash); health gate is `npm run typecheck · npm run test · npm run build`.

0. The model in one paragraph

Three co-equal systems form a triangle. **R&D lines** accumulate *tech levels* from allocated budget. Tech levels **gate** which **product archetypes** can be created and set their *quality*. Creating/upgrading a product is a **bet** (the signature mechanic, now multiple-in-parallel) that consumes **capacity** while it runs and ships a live product that *keeps* consuming capacity and grows organically (ramp → mature → decay). Revenue funds more of all three. Everything below is the mechanics that make that loop run, deterministically, off the authored TOML.

1. New content schemas & loaders

Four new content shapes. Mirror the existing loader pattern (`src/content/load.ts` glob + `smol-toml` → typed views in `src/domain/content.ts` → id-indexed maps on `ContentDB`).

1.1 Folders / glob

<code>content/rd_lines/<sub_industry>.toml</code>	→ <code>RDLine[]</code> (keyed tables, one per line)
<code>content/products/<sub_industry>.toml</code>	→ <code>ProductArchetype[]</code> (keyed tables)
<code>content/capacity/<sub_industry>.toml</code>	→ <code>CapacityType[]</code> (keyed tables, each)
<code>content/products/_tuning.toml</code>	→ <code>ProductTuning</code> by <code>sub_industry</code>

Add four globs alongside the existing ones. All resolve at build (dev-server restart to pick up new files), same as today.

1.2 Types (mirror TOML 1:1 — `src/domain/content.ts`)

```
type SpecTag = string; // free-form per sub-industry, but consistent within it

interface RDLine {
  id: string;
  sub_industry: SubIndustry;
  name: string;
  description: string;
  icon?: string;
  starting_level: number;           // 0-100+
  base_cost_per_quarter: number;    // $M anchor cost to advance ~1 level at low
  drives_specs: SpecTag[];
}

interface ProductArchetype {
  id: string;
  sub_industry: SubIndustry;
  name: string;
  tier: number;                     // 1..N, contiguous
  description: string;
  flavor_naming_hint?: string;
  gates: Record<string /*lineId*/, number /*required tech level*/>;
  economics: {
    build_cost: number;             // $M, one-time bet cost
    build_weeks: number;            // bet duration
    unit_margin: number;            // 0-1 gross margin at launch
    capacity_type: string;          // capacity id in same sub-industry
    capacity_to_build: number;      // units tied up DURING the bet
    capacity_to_run: number;        // units a SHIPPED product occupies while li
    addressable_market: number;     // $M/yr ceiling for this product class
    ramp_weeks: number;             // ship → peak revenue
    decay_per_quarter: number;      // fraction; obsolescence rate once mature
  };
  specs: Record<SpecTag, number>;   // WEIGHTS, sum ≈ 1.0
}

interface CapacityRung { capacity: number; cost: number; build_weeks: number; }
interface CapacityType {
  id: string;
  sub_industry: SubIndustry;
  name: string;
  unit_label: string;
```

```

    description: string;
    rungs: CapacityRung[];           // ≥6, monotonic increasing in all three fie
}

interface ProductTuning {
    starting_capacity: number;
    rd_diminishing_k: number;       // 0-1; higher = harsher diminishing returns
    frontier_pull: number;          // ≥1; R&D progress bonus when behind the fr
    max_concurrent_bets: number;
    build_cost_mult: number;
    build_time_mult: number;
    decay_mult: number;
    share_volatility: number;       // how fast share can swing vs rivals
}

```

Expose on `ContentDB`: `rdLinesBySub`, `productsBySub`, `capacityBySub`,
`productTuningBySub`, plus flat `...ById` maps.

1.3 Load-time validation (push to `ContentDB.warnings`, don't crash)

Implement exactly the checks already proven against the authored content:

1. Each product `gates` key resolves to a real `RDLine.id` in the **same** sub-industry.
2. Each product `economics.capacity_type` resolves to a real `CapacityType.id` in the same sub-industry.
3. Each product `specs` weights sum to $1.0 \pm 1e-6$.
4. Each product `specs` tag appears in **some** line's `drives_specs` for that sub-industry.
5. Each `CapacityType.rungs` length ≥ 6 and strictly increasing in `capacity`, `cost`, and non-decreasing in `build_weeks`.
6. Tiers per sub-industry are contiguous $1..N$; `build_cost` and `build_weeks` non-decreasing by tier.
7. `capacity_to_build` $\leq \max(\text{rung.capacity})$ for the product's capacity type (else unbuildable).
8. All six sub-industries have a `_tuning` block.

A reference implementation of 1–8 (Python) was used to validate the authored content and returned zero errors — port the same assertions into the loader, plus a unit test (`products.content.test.ts`) asserting the shipped content is clean.

Known soft warnings (expected, not failures): some lines declare a `drives_specs` tag no

product currently weights (ai_chips : cost , specialization ; launch_services : margin ; satellite_constellations : margin ; space_stations : reliability). These are balance items, not errors — the loader may log them at a lower severity.

2. State (per company, in the run state)

Add to the operating-company state (serialized in saves):

```
interface RDState {
    levels: Record<string /*lineId*/, number>; // current tech level per line
    allocation: Record<string /*lineId*/, number>; // player's budget split, fraction
    rd_budget_per_week: number; // $M/wk the player commits to
}

interface CapacityState {
    // per capacity type: which rungs are built, and in-progress rung builds
    owned: Record<string /*capId*/, number>; // total capacity units available
    rung_index: Record<string /*capId*/, number>; // highest rung purchased (-1 =
    builds_in_progress: Array<{ cap_id: string; rung_index: number; weeks_left: number }>
}

interface ActiveBet {
    id: string; // run id (seeded)
    archetype_id: string;
    instance_name: string; // player-named
    kind: "create" | "upgrade";
    weeks_left: number;
    capacity_held: number; // = archetype.capacity_to_build (released on ship)
    cap_id: string;
    // snapshot of tech levels at commit, for resolution quality
    committed_levels: Record<string, number>;
}

interface LiveProduct {
    id: string; // instance id (seeded)
    archetype_id: string;
    instance_name: string;
    shipped_week: number;
    quality: number; // 0-100, computed at ship (see §3.3); upgrades raise
    age_weeks: number;
    state: "ramping" | "mature" | "declining";
    share: number; // 0-1 of addressable_market, evolves vs rivals
    revenue_run_rate: number; // $M/yr, derived; cached for UI
    capacity_run: number; // = archetype.capacity_to_run
```

```

}

interface ProductsRuntime {
  rd: RDState;
  capacity: CapacityState;
  bets: ActiveBet[];
  products: LiveProduct[];
}

```

3. The math (pure functions in `src/engine/`)

All deterministic; any randomness draws from the seeded RNG already in the codebase.

3.1 R&D progress (`src/engine/rd.ts`)

Each advanced week, for each line with allocation > 0:

```

spend_wk      = rd_budget_per_week * allocation[line]
# anchor: base_cost_per_quarter advances ~1 level/qtr at low levels.
# normalize to weekly: base_cost_per_week = base_cost_per_quarter / 13
raw_gain      = spend_wk / base_cost_per_week          # levels/week at full
# diminishing returns as level climbs:
dim           = (1 - rd_diminishing_k * (level/100))    # clamp ≥ 0.1
# frontier pull: if rivals' frontier in this line's specs is ahead, boost catch-u
gap           = max(0, rival_frontier_level - level)
pull         = 1 + (frontier_pull - 1) * clamp(gap/40, 0, 1)
delta_level   = raw_gain * dim * pull
level        = min(LEVEL_CAP, level + delta_level)     # LEVEL_CAP e.g. 100

```

- `rival_frontier_level` for a line = derive from the sub-industry's anchor companies' `quality` block (you already have `fundamentals / execution / benchmark_score`); pick the mapping that fits each line's `drives_specs` . Keep it simple: the strongest rival's relevant quality, scaled to the 0–100 line space.
- `rd_diminishing_k` , `frontier_pull` come from `_tuning[sub]` .
- **Determinism:** pure arithmetic; no RNG needed here.

3.2 Capacity accounting (`src/engine/capacity.ts`)

```

available(cap_id) = owned[cap_id]
                    - ∑ bets.capacity_held where bet.cap_id==cap_id
                    - ∑ products.capacity_run where product's archetype.capacity_ty

```

- Starting `owned = _tuning[sub].starting_capacity` (base, before any rung).
- Buying rung `i` schedules a `builds_in_progress` entry; on completion `owned += rung.capacity` (the rung values in TOML are **deltas added**, matching the authored ladders).
- A new bet may start only if `available(cap_id) ≥ archetype.capacity_to_build` **and** `bets.length < max_concurrent_bets`.

3.3 Product quality at ship (`src/engine/products.ts`)

```
for each spec tag in archetype.specs:
    # the lines that drive this tag:
    driving = lines where tag ∈ line.drives_specs
    line_score = average(committed_levels[l] for l in driving)    # 0-100
    spec_score[tag] = line_score
quality = Σ archetype.specs[tag] * spec_score[tag]              # weighted, 0-10
```

- Upgrades re-run this with current levels and take `max(old_quality, new)` (or blend) so an upgrade bet meaningfully raises the product.

3.4 Market share vs rivals (`src/engine/productMarket.ts`)

Per advanced week (or per quarter close — your call, but keep it deterministic):

```
your_q    = product.quality
rival_q   = representative rival quality in this product class (from anchor cos in
target    = logistic( (your_q - rival_q) / spread )                # 0-1 target share
# move current share toward target, rate limited by volatility:
product.share += clamp(target - product.share, -v, +v)            # v = share_volati
```

- `share_volatility` from `_tuning[sub]`. Sticky industries (SaaS, launch) barely move; hype ones (frontier lab) swing.
- Optional hype coupling: nudge `target` by the sector hype band for high-hype_exposure classes (reuse existing sector hype state).

3.5 Revenue, ramp, decay (`src/engine/products.ts`)

```
peak_rev_run_rate = product.share * archetype.addressable_market    # $M/yr at ful
# ramp: linear/eased from ship over ramp_weeks
ramp_frac = clamp(age_weeks / ramp_weeks, 0, 1)
# decay once mature: after ramp, apply decay_per_quarter * decay_mult
```

```

if state == declining:
    decay_wk = (archetype.decay_per_quarter * decay_mult) / 13
    decay_multiplier *= (1 - decay_wk)
revenue_run_rate = peak_rev_run_rate * ramp_frac * decay_multiplier
gross_profit      = revenue_run_rate * archetype.unit_margin * (margin decays slig

```

- **State machine:** `ramping` (age < `ramp_weeks`) → `mature` (a plateau window) → `declining` (decay applies). Decay rate is per-archetype × `decay_mult`.
- This is what makes revenue grow **between** bets (fixes bug #3): a live product's `revenue_run_rate` rises across its ramp every week with no new event.
- Feed `Σ products.gross_profit` into the existing finance/earnings pipeline as operating revenue from products.

3.6 Bets / the signature mechanic, now concurrent (`src/engine/signature.ts` — extend, don't replace)

- Committing a bet = choosing an archetype (gates must be met), naming the instance, paying `build_cost * build_cost_mult`, reserving `capacity_to_build`, setting `weeks_left = round(build_weeks * build_time_mult)`.
- Multiple bets run in parallel up to `max_concurrent_bets` and capacity. Each ticks down weekly; on `weeks_left==0` it **ships**: release build capacity, create a `LiveProduct` (reserving `capacity_to_run`), compute quality (§3.3), enter `ramping`.
- The existing per-sub-industry signature noun (`training run / tape-out / launch`) is now the **label** for “a bet in flight”; resolution odds/flavor can stay, but the outcome is “product ships” not a one-off effect.
- Keep it deterministic: any success/variance roll uses the seeded RNG keyed by the bet id.

4. The operating loop wiring

Per advanced week, in deterministic order (extend `src/engine/tick.ts`):

1. Apply R&D progress (§3.1) → update tech levels.
2. Tick capacity builds (§3.2) → completed rungs add capacity.
3. Tick active bets (§3.6) → ship completed products.
4. Update live products: age, state transitions, share (§3.4), revenue/decay (§3.5).

5. Roll product gross profit into finance/earnings (existing pipeline).

6. Recompute `available(cap)` for UI.

Budget competition is the core decision each turn: the player sets `rd_budget_per_week` and the per-line `allocation`, decides whether to buy a capacity rung, and whether to commit a new bet — all competing for the same cash with go-to-market/ops/treasury. No new “spend ceiling”: capacity rungs + new product tiers + new lines of business mean there’s always a next thing to fund.

5. UI surfaces (apply `UI_LANGUAGE.md` — borderless, alive, no dead space)

Three new/extended surfaces, all in the established visual language:

- **R&D allocation panel** — the lines as horizontal bars/sliders summing the R&D budget; each shows tech level, a live “drives: performance, efficiency” caption, and a *creep* animation while funded (continuous motion). The “vs.” is visible: moving one slider visibly steals from others.
- **Product portfolio** — live products as borderless hairline rows: name, archetype, quality, share-vs-rivals mini-bar, revenue run-rate (counts up on advance), lifecycle chip (ramping/mature/declining). In-flight **bets** shown the same way with a progress creep + the signature noun (“training run · 61% · ~7 wks”).
- **Capacity meter** — current `available` / `owned`, the next rung’s cost/time, and what’s consuming capacity right now (bets + live products). The “buy next rung” is the ongoing-spend replacement for the old three buys.

Concurrency is the headline feel: mid-game this screen shows several bets in flight, a portfolio of products at different lifecycle stages, R&D lines creeping, and capacity as the shared constraint. That’s the depth made visible.

6. Content/code boundary (so it stays maintainable)

- **Content (TOML):** archetypes, lines, capacity ladders, per-industry tuning — *what exists and the per-industry economics* (timing, costs, decay, margins, gates).
Adding/balancing a sub-industry = authoring only.
- **Code (engine):** the allocation math, gating, quality computation, share dynamics, ramp/decay state machine, bet scheduler/concurrency, capacity accounting, and UI.

How the world reacts — identical across all six sub-industries.

- The line between them is the whole maintainability bet: a seventh sub-industry (a future DLC industry) ships as four TOML files, no engine change.
-

7. Tests to add (content-contract + engine)

- `products.content.test.ts` — runs the §1.3 validation over shipped content; asserts zero errors (the four soft warnings allowed).
 - `rd.test.ts` — diminishing returns monotonic; frontier_pull boosts when behind; deterministic given seed + allocation.
 - `capacity.test.ts` — `available()` accounting; rung completion adds the authored delta; bet blocked when capacity short or at `max_concurrent_bets`.
 - `productLifecycle.test.ts` — ramp raises revenue between bets (the bug-#3 regression guard); decay reduces it after maturity; quality from tech levels × spec weights matches a hand-worked example.
 - `signatureConcurrency.test.ts` — N parallel bets ship independently; capacity released on ship.
-

8. Bug fixes (separate from the feature — but bundle in the same branch)

BUG 1 — IPO → “fundraise” reverts company to private, lets you IPO again

Symptom: after going public, opening the raise flow sets `status` back to `private`; the IPO action re-enables and can fire a second time. **Cause:** the raise/fundraise path unconditionally writes `status = "private"` (or the IPO action’s enable check only tests `can_raise`, not current `status`). **Fix:**

- The fundraise flow for a **public** company must route to a follow-on/secondary path (or be disabled), never reset `status`. Guard the status write: only the IPO completion transitions `private → public`; nothing transitions `public → private` except an explicit go-private feature (not in V1).
- Gate the IPO action on `status == "private"` (and the existing readiness checks). Add `lockup / weeks_public` awareness if a secondary raise should be allowed only post-

lookup. **Regression test:** `ipoStateMachine.test.ts` — after IPO, `status==public` ; invoking `fundraise` does not change `status` ; IPO action is unavailable.

BUG 2 — earnings popup doesn't reflect the new earnings-management fields

Symptom: the earnings popup renders the old shape; the guidance / engineered-vs-real fields we designed don't appear. **Cause:** the popup component reads the legacy earnings model, not the new fields (guidance set, beat/miss, engineered vs. real/normalized figures).

Fix:

- Point the popup at the current earnings object; render the new fields: reported vs. guidance (beat/meet/miss), real vs. engineered/normalized figures, and the market reaction tied to the new model.
- If the new fields are computed but not threaded into the popup's props, thread them; if not yet computed at quarter close, compute in the earnings step and store on the report object the popup consumes. **Regression test:** `earningsReport.test.ts` — a quarter close with guidance + an engineered beat produces a report object containing the new fields, and the popup selector reads them.

BUG 3 — chip-maker revenue doesn't grow between tape-outs

Status: fixed by this feature. Revenue was event-stepped (jumped only when a chip shipped). With §3.5, every live product's `revenue_run_rate` ramps weekly across `ramp_weeks` and evolves with market share — so revenue grows continuously in the gaps, not just at ship. **Action:** ensure the legacy "revenue only changes on signature resolution" path is removed for product-driven sub-industries once products are wired; `productLifecycle.test.ts` (the ramp-between-bets assertion) is the guard.

9. Suggested build order

1. Schemas + loaders + §1.3 validation + `products.content.test.ts` (prove the content loads clean).
2. R&D math + capacity accounting + their tests (no UI yet).
3. Bet scheduler/concurrency + lifecycle/revenue + tests (the loop runs headless).
4. Wire product gross profit into finance/earnings; remove legacy stepped-revenue path (closes bug #3).
5. UI surfaces (allocation, portfolio, capacity).

6. Bugs #1 and #2 (independent; can land anytime in the branch).
7. Balance pass once playable — start with the four soft-warning spec tags.